

WRO Engineering Journal

Team Information

Team Name:	SSTechnitum
Team ID:	[Enter Team ID]
School:	School of Science and Technology
Category:	WRO Future Engineers
Team Members:	[Zwe Paing Htet] — [Lead Builder] [Jayrius Kee] — [Lead Coder]

Team Photo

Hardware Overview

Table of components used by the robot:

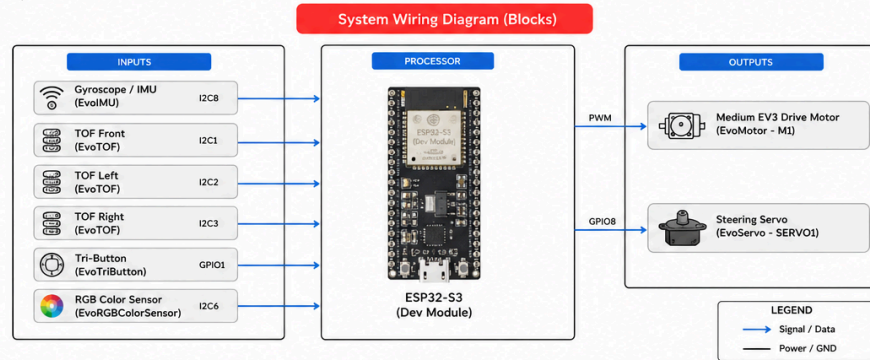
Part Name	Model / Quantity	Qty
Microcontroller	EvolutionX1, ESP32-S3	1
Motor (Drive)	EV3 Medium Motor	1
Steering Servo	GeekServo 360° Grey	1
Camera	MaixCam	1
Time of Flight	VL53L0X	3
IMU / Gyroscope	BNO055	1
RGB Colour Sensor	EvoRGBColorSensor	1
Start/Reset Button	EvoTriButton	1
I2C Multiplexer	TCA9548A	1
IO Multiplexer	SX1506	1
Battery	NCR18650B	2

Diagrams

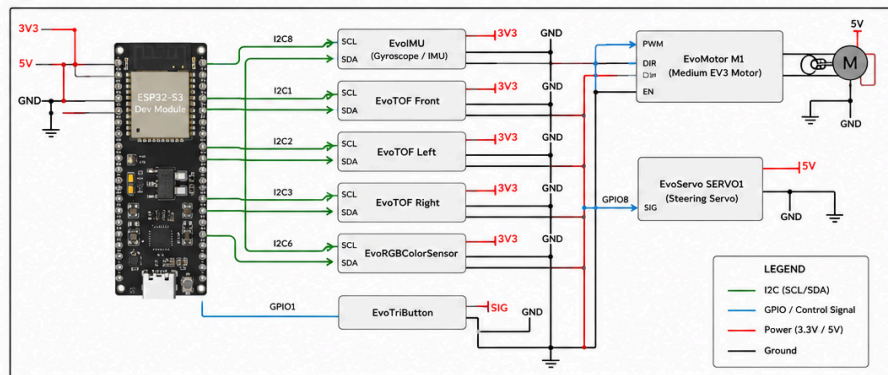
Microcontroller System Diagram

Diagrams

System Wiring Diagram (Blocks)



Microcontroller System Diagram (Schematic)



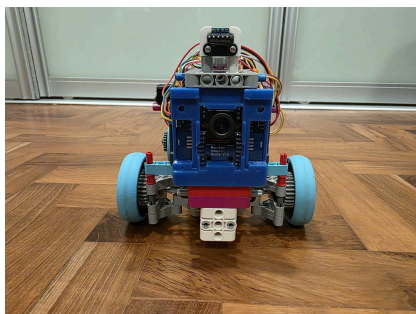
Drivetrain

Chassis Design

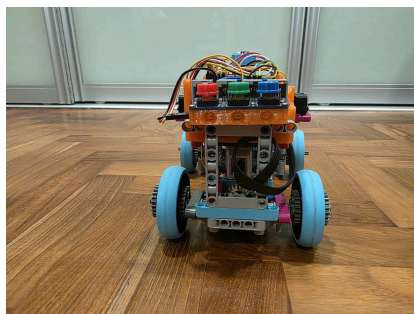
The chassis is built entirely out of LEGO parts. This lets us modify and iterate the design quickly whenever changes are needed, rather than having to redesign and reprint or recut structural parts.

Photos of the vehicle:

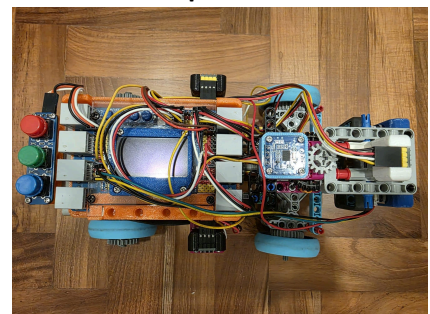
Front View

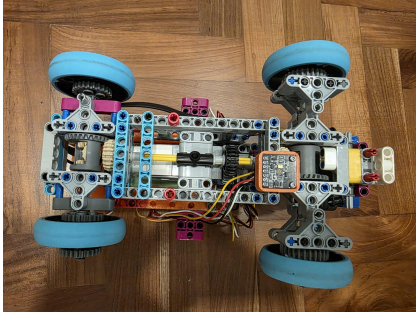
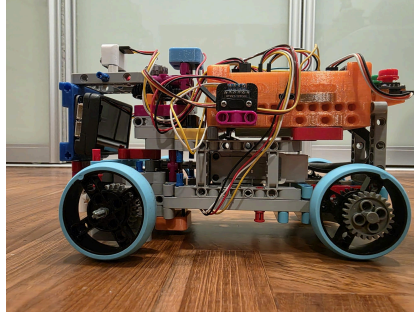
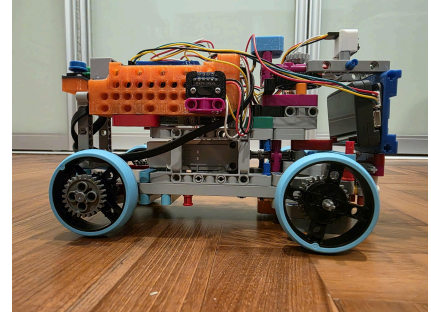


Back View



Top View



Bottom View**Left View****Right View**

Motors

Drive power comes from an EV3 Medium Motor, which powers a dual differential system driving both rear wheels. We also considered N20 motors, which are smaller and would have allowed for a more compact design, but ruled them out because they don't provide enough torque to drive the robot reliably.

Differential

A dual differential is used on the rear axle, allowing both rear wheels to rotate at different speeds so the robot can corner smoothly instead of dragging or skidding a wheel through turns.

We also prototyped a dual differential steering system, where both front wheels would turn together through a differential linkage. This achieved a noticeably tighter turning radius, but introduced too much friction at the wheels — the steering servo wasn't strong enough to reliably push through that friction and reach an accurate turning angle, so the design was dropped in favour of our current servo-and-linkage steering setup.

Steering

Steering is handled by a single servo connected to a LEGO beam, which links to the wheels through an L-shaped beam. This direct servo-driven linkage replaced our earlier dual differential steering attempt and gave us reliable, accurate steering without the excess friction of that design.

Components

Evolution X1

The Evolution X1 is our main controller board, built around an ESP32-S3 microcontroller and initialised in code through the Evo library (`evo.begin()`). It handles sensor polling, the OLED status display, and motor/servo output every loop, and exposes the I2C bus addresses (I2C1–I2C8) used to connect our TOF sensors, IMU, and colour sensor.

Time of Flight (ToF) Sensors

Three VL53L0X ToF sensors (EvoTOF) are mounted facing front, left, and right, each running in long-range mode (`setLongRange(true)`) for extended detection distance. Each reading is offset by a fixed calibration value (-45 front, -62 left, -68 right) to correct for the sensor's physical mounting position relative to the chassis edge.

Gyroscope (IMU)

A BNO055 IMU (EvoIMU) tracks the robot's relative heading. On startup, `gyro.resetHeading()` zeroes the heading once the start button is pressed. During driving, the current heading is compared against the target heading, and the error is fed into a proportional controller ($k_p = 0.75$) that adjusts the steering servo angle to keep the robot driving straight.

Drive Motor

Drive power comes from a single EV3 Medium Motor (EvoMotor), connected at port M1. It is controlled purely by a speed value passed to `motor.run()`, independent of the steering servo, so drive speed and steering angle can be adjusted separately.

Steering Servo

Steering is handled by a GeekServo 360° Grey servo (EvoServo) on port SERVO1/GPIO8, running at a standard 50Hz signal. The centre position is 95°, and the gyro-correction output is clamped between 60° and 130° so the servo never over-rotates and jams the steering linkage.

RGB Colour Sensor

An EvoRGBColorSensor on I2C6 reads red, green, and blue channel values every loop. These are compared against calibrated RGB ranges to identify red and green obstacle blocks, which the robot uses to decide which side to pass an obstacle on.

Start/Reset Button

An EvoTriButton on GPIO1 is used to start the run: `triButton.waitForBump()` pauses the program until the button is pressed, at which point the gyro heading is reset so the run always starts from a heading of 0°.

Huskly lens

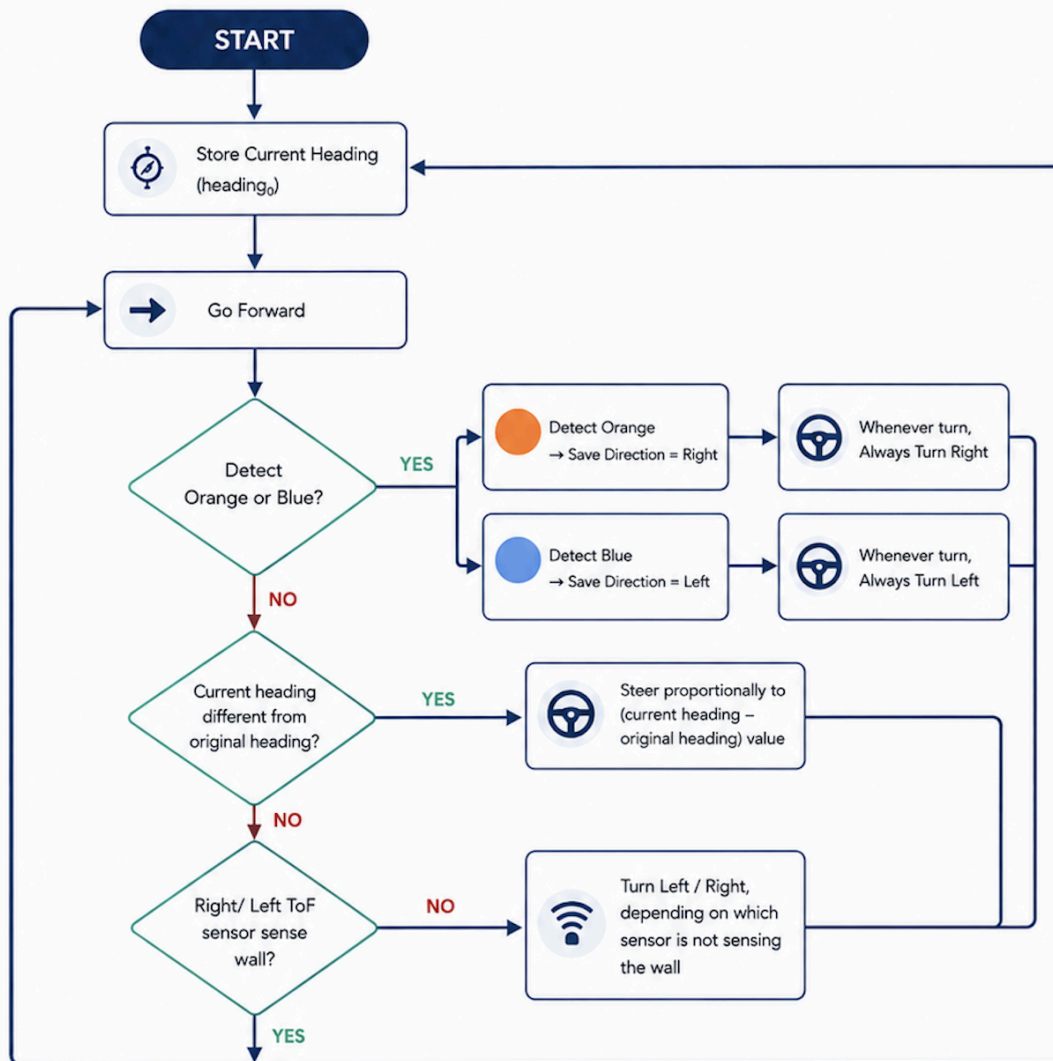
The HuskyLens is a compact AI vision camera that runs machine-learning models directly on the module, letting it detect and classify objects in real time without sending frames to an external computer. On our robot, it looks for the red and green obstacle blocks: when it detects a red block, the robot turns slowly to the left to pass it; when it detects a green block, the robot turns slowly to the right.

Strategy



STRATEGY

PRELIMINARY ROUND FLOWCHART



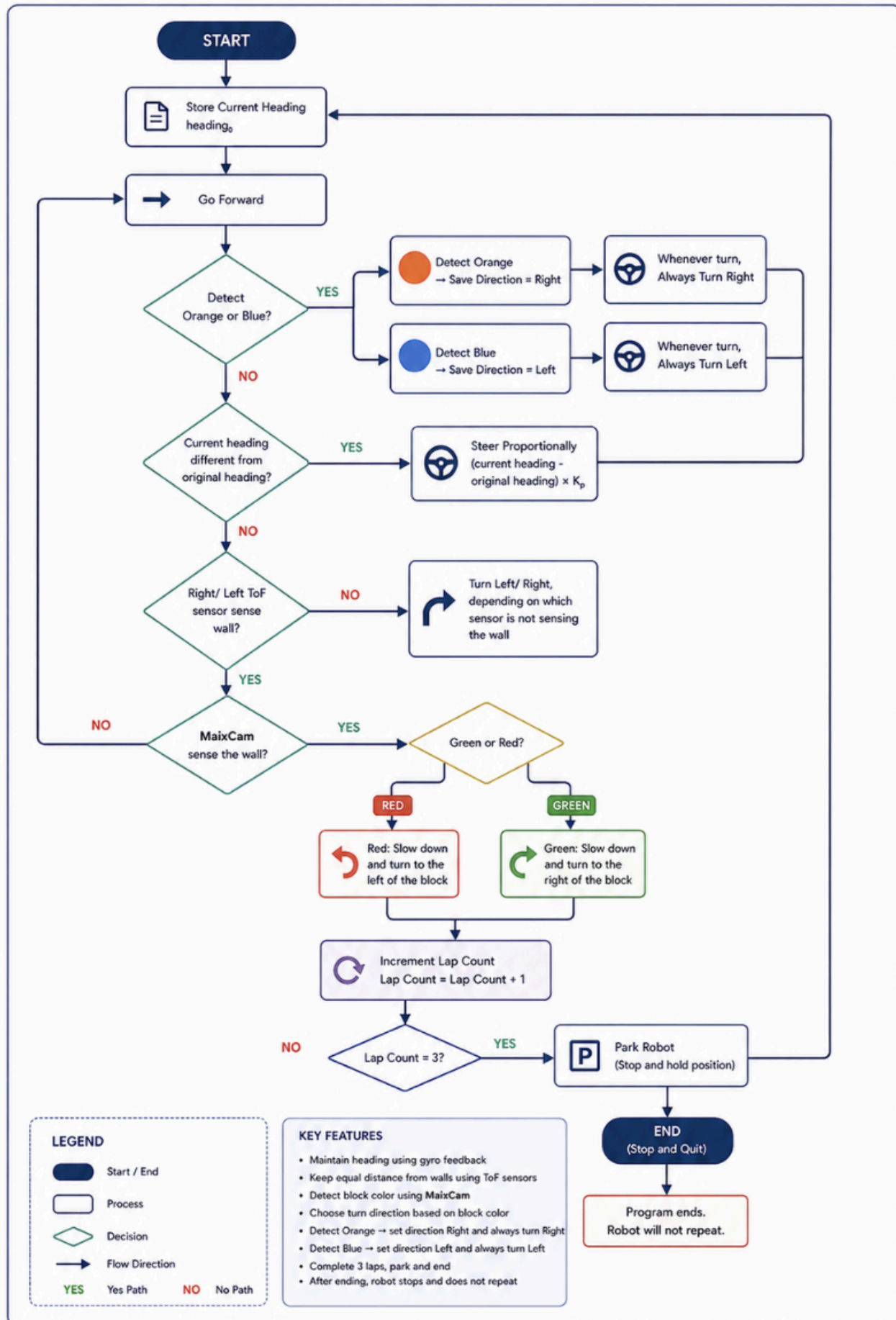
LEGEND

	Start / End		Decision	YES	Yes Path
	Process		Flow Direction	NO	No Path

KEY FEATURES

- Maintain initial heading using gyro feedback
- Move forward while correcting heading drift
- Use ToF sensors to detect walls on left or right
- Turn toward the open side if a wall is detected
- Detect Orange → set direction Right and always turn Right
- Detect Blue → set direction Left and always turn Left

OBSTACLE ROUND FLOWCHART



Code

Keep the codebase readable and easy for every team member to follow. Even though the self-driving logic is complex, break it into smaller parts with clear labels.

Guidelines for clean code:

- Write comments at key points to explain what each section does.
- Use libraries and separate classes to keep the main file simple and uncluttered.
- Give variables and functions clear, meaningful names.

No Obstacles Code:

```
#include <Evo.h>
#include <ESP32Servo.h>

// =====
// PRELIMINARY ROUND FLOWCHART
// =====
//
// START
// -> Store Current Heading (heading0)
// -> Go Forward
// -> Detect Orange or Blue?
//   ORANGE -> Save Direction = Right -> Always Turn Right
//   BLUE   -> Save Direction = Left  -> Always Turn Left
//   NO     -> Is Current Heading Different From Original?
//     YES -> Steer Proportionally
//     NO  -> Do Left/Right ToF Sensors Detect Wall?
//       NO -> Turn Toward the Open Side
//       (or Locked Color Direction)
//       YES -> Continue Forward
//
// Every TURN or STEERING CORRECTION updates heading0 before the next
// loop, returning to "Store Current Heading" in the flowchart.
//
// If both side sensors detect walls, the robot continues forward
// without updating heading0.
//
// NOTE:
// The flowchart mentions MAIXCAM, but this code currently uses HuskyLens
// for the ID-2 stop feature. Orange/Blue detection uses the onboard RGB
// color sensor.
// =====

EVOX1 evo;

// --- IMU ---
EvoIMU gyro(I2C8);

// --- Servo ---
```



```

Servo steering;

// --- ToF Sensors ---
EvoTOF tofFront(I2C1);
EvoTOF tofLeft(I2C2);
EvoTOF tofRight(I2C3);

// --- Motor ---
EvoMotor motor(M1, EV3MediumMotor, true);

// --- Button ---
EvoTriButton triButton(GPIO1);

// --- RGB Color Sensor ---
EvoRGBColorSensor colorSensor(I2C6);

// --- HuskyLens ---
EvoHuskyLens husky(I2C7);

// =====
// TUNABLE CONSTANTS
// =====

// Motor / Steering
const int FORWARD_SPEED = -3000; // Forward motor speed
const int SERVO_CENTER = 95; // Straight-ahead servo position
const int TURN_SERVO_OFFSET = 35; // Servo offset used during turns
const int STEER_MAX_OFFSET = 25; // Maximum steering correction

// Heading Correction
const float HEADING_THRESHOLD_DEG = 3.0; // Heading error deadband
const float STEER_KP = 1.2; // Steering correction strength

// Wall Detection
const int WALL_DETECT_DISTANCE_MM = 250; // Below this = wall detected

// Turn Execution
const float TURN_ANGLE_DEG = 90.0; // Target angle for each turn

// Color Detection
// Adjust these values using the Serial Monitor with your actual markers.
const int ORANGE_R_MIN = 150;
const int ORANGE_B_MAX = 90;

const int BLUE_B_MIN = 150;
const int BLUE_R_MAX = 90;

// HuskyLens Stop Object
// Stops the robot when ID 2 is detected at the required size.
const int STOP_ID = 2;

```

```

const int MIN_WIDTH = 25;
const int MIN_HEIGHT = 35;

// =====
// DIRECTION / COLOR CODES
// =====

enum Direction
{
    DIR_NONE = 0,
    DIR_LEFT = -1,
    DIR_RIGHT = 1
};

enum ColorSeen
{
    COLOR_NONE = 0,
    COLOR_ORANGE = 1,
    COLOR_BLUE = 2
};

// =====
// STATE
// =====

float heading0 = 0;          // Stored reference heading
int lockedDirection = DIR_NONE; // Direction locked by orange/blue marker

int distFront;
int distLeft;
int distRight;

int red;
int green;
int blue;

// =====
// HELPER FUNCTIONS
// =====

// Keeps an angle difference within the range -180 to +180 degrees.
float normalizeAngle(float angleDeg)
{
    while (angleDeg > 180)
        angleDeg -= 360;

    while (angleDeg < -180)
        angleDeg += 360;

    return angleDeg;
}

```

```

}

// Reads the ToF and RGB sensors.
void readSensors()
{
    distFront = tofFront.getDistance() - 45;
    distLeft = tofLeft.getDistance() - 62;
    distRight = tofRight.getDistance() - 68;

    red = colorSensor.getRed();
    green = colorSensor.getGreen();
    blue = colorSensor.getBlue();
}

// Detects whether the RGB sensor sees an orange or blue marker.
int classifyColor(int r, int g, int b)
{
    if (r > ORANGE_R_MIN && b < ORANGE_B_MAX)
        return COLOR_ORANGE;

    if (b > BLUE_B_MIN && r < BLUE_R_MAX)
        return COLOR_BLUE;

    return COLOR_NONE;
}

// Checks HuskyLens for the ID-2 stop object.
// Returns true when the robot should stop permanently.
bool checkStopObject()
{
    HUSKYLENSResult result;

    if (husky.requestBlocks(result))
    {
        Serial.print("ID=");
        Serial.print(result.ID);

        Serial.print(" W=");
        Serial.print(result.width);

        Serial.print(" H=");
        Serial.println(result.height);

        if (result.ID == STOP_ID &&
            result.width >= MIN_WIDTH &&
            result.height >= MIN_HEIGHT)
        {
            return true;
        }
    }
}

```

```

    return false;
}

// Performs a turn in the selected direction.
// The robot keeps moving forward until the gyro detects the target
// turning angle, then the steering returns to the center.
void executeTurn(int dir)
{
    float startHeading = gyro.getHeading();

    int servoAngle =
        (dir == DIR_RIGHT)
        ? (SERVO_CENTER + TURN_SERVO_OFFSET)
        : (SERVO_CENTER - TURN_SERVO_OFFSET);

    steering.write(servoAngle);

    while (true)
    {
        motor.run(FORWARD_SPEED);

        float diff =
            normalizeAngle(gyro.getHeading() - startHeading);

        if (dir == DIR_RIGHT && diff >= TURN_ANGLE_DEG)
            break;

        if (dir == DIR_LEFT && diff <= -TURN_ANGLE_DEG)
            break;

        delay(10);
    }

    steering.write(SERVO_CENTER);
}

// Corrects the robot's heading proportionally to the heading error.
void steerProportional(float headingDiff)
{
    int offset = (int)(STEER_KP * headingDiff);

    offset = constrain(
        offset,
        -STEER_MAX_OFFSET,
        STEER_MAX_OFFSET
    );

    steering.write(SERVO_CENTER + offset);
}

```

```

// Displays the current robot status and sensor readings.
void showStatus(const char* line1, const char* line2)
{
    evo.clearDisplay();

    evo.writeToDisplay(line1, 0, 0, true, false);
    evo.writeToDisplay(line2, 0, 16);

    evo.writeToDisplay("L:", 0, 32);
    evo.writeToDisplay(distLeft, 16, 32);

    evo.writeToDisplay("R:", 45, 32);
    evo.writeToDisplay(distRight, 61, 32);

    evo.writeToDisplay("F:", 90, 32);
    evo.writeToDisplay(distFront, 106, 32);

    evo.writeToDisplay("Dir:", 0, 48);

    evo.writeToDisplay(
        lockedDirection == DIR_RIGHT ? "R" :
        lockedDirection == DIR_LEFT ? "L" :
        "-",
        30,
        48
    );

    evo.drawDisplay();
}

// =====
// SETUP
// =====

void setup()
{
    Serial.begin(115200);

    // Start EVO and IMU.
    evo.begin();
    gyro.begin();

    // Start and configure ToF sensors.
    tofFront.begin();
    tofFront.setLongRange(true);

    tofLeft.begin();
    tofLeft.setLongRange(true);
}

```

```

tofRight.begin();
tofRight.setLongRange(true);

// Start motor.
motor.begin();

// Configure steering servo.
steering.setPeriodHertz(50);
steering.attach(GPIO8, 1000, 2000);
steering.write(SERVO_CENTER);

// Start RGB color sensor.
colorSensor.begin();

// Start HuskyLens.
if (!husky.begin())
{
    evo.clearDisplay();

    evo.writeToDisplay(
        "HUSKY ERROR",
        0,
        0,
        true,
        false
    );

    evo.writeToDisplay(
        "CHECK I2C7",
        0,
        16
    );

    evo.drawDisplay();

    // Stay here if HuskyLens fails to start.
    while (true)
    {
        delay(1000);
    }
}

husky.setMode(ALGORITHM_COLOR_RECOGNITION);

// Show ready screen.
evo.clearDisplay();

evo.writeToDisplay(
    "READY",
    0,

```



```

    0,
    true,
    false
);

evo.writeToDisplay(
    "PRESS BUTTON",
    0,
    16
);

evo.drawDisplay();

// Wait for the start button.
triButton.waitForBump(3);

// Reset and store the initial heading.
gyro.resetHeading();
delay(200);

heading0 = gyro.getHeading();
}

// =====
// MAIN LOOP
// =====

void loop()
{
    // Check for the HuskyLens stop object.
    if (checkStopObject())
    {
        motor.stop();

        showStatus(
            "ID 2 FOUND!",
            "STOPPED"
        );

        // Stop permanently.
        while (true)
        {
            delay(1000);
        }
    }

    // -----
    // GO FORWARD
    // -----

```

```

motor.run(FORWARD_SPEED);
readSensors();

// -----
// DETECT ORANGE OR BLUE
// -----

int seenColor = classifyColor(
    red,
    green,
    blue
);

if (seenColor == COLOR_ORANGE)
{
    // Orange locks the robot to right turns.
    lockedDirection = DIR_RIGHT;

    executeTurn(DIR_RIGHT);

    heading0 = gyro.getHeading();

    showStatus(
        "ORANGE SEEN",
        "TURN RIGHT"
    );

    return;
}

if (seenColor == COLOR_BLUE)
{
    // Blue locks the robot to left turns.
    lockedDirection = DIR_LEFT;

    executeTurn(DIR_LEFT);

    heading0 = gyro.getHeading();

    showStatus(
        "BLUE SEEN",
        "TURN LEFT"
    );

    return;
}

// -----
// CHECK HEADING
// -----

```

```

float currentHeading = gyro.getHeading();

float headingDiff =
    normalizeAngle(currentHeading - heading0);

if (fabs(headingDiff) > HEADING_THRESHOLD_DEG)
{
    steerProportional(headingDiff);

    // Store the corrected heading for the next loop.
    heading0 = gyro.getHeading();

    showStatus(
        "CORRECTING",
        "HEADING"
    );

    return;
}

// -----
// CHECK SIDE WALLS
// -----

bool leftWallSensed =
    (distLeft < WALL_DETECT_DISTANCE_MM);

bool rightWallSensed =
    (distRight < WALL_DETECT_DISTANCE_MM);

// If either side is open, turn toward the open side.
if (!leftWallSensed || !rightWallSensed)
{
    int dir;

    // If orange/blue has already locked a direction,
    // always use that direction.
    if (lockedDirection != DIR_NONE)
    {
        dir = lockedDirection;
    }
    else
    {
        // Otherwise, turn toward the side without a wall.
        dir = (!leftWallSensed)
            ? DIR_LEFT
            : DIR_RIGHT;
    }
}

```

```

    executeTurn(dir);

    heading0 = gyro.getHeading();

    showStatus(
        "WALL OPENING",
        dir == DIR_RIGHT
        ? "TURN RIGHT"
        : "TURN LEFT"
    );

    return;
}

// -----
// BOTH SIDES HAVE WALLS
// -----

// Continue driving straight.
// heading0 is intentionally left unchanged.
steering.write(SERVO_CENTER);

showStatus(
    "DRIVING",
    "STRAIGHT"
);
}

```

- END -